

PAULT DOCUMENTATION

2026 01 27 Ui Overhaul Implementation

Source: docs/plans/2026-01-27-ui-overhaul-implementation.md

Theme: Clean Technical

Generated: February 26, 2026

UI Overhaul Implementation Plan

> **Status: COMPLETED** — All phases and tasks implemented successfully.

> **For Claude:** REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task.

Goal: Transform Pault from a fixed-layout prompt manager into a native macOS app with auto-hiding sidebar, tag-based organization, always-editable content, and collapsible inspector.

Architecture: Replace the manual HStack layout with NavigationSplitView. Replace Category (one-to-many) with Tag (many-to-many). Extract sidebar, inspector, and tag pill into separate view files. Remove edit mode—content is always editable with auto-save.

Tech Stack: SwiftUI, SwiftData, macOS 14+

Phase 1: Data Model

Task 1: Create Tag Model

Files:

- Create: Pault/Tag.swift
- Test: PaultTests/TagTests.swift

Step 1: Write the failing test

Create PaultTests/TagTests.swift:

```
//  
// TagTests.swift  
// PaultTests  
//  
  
import Testing  
import SwiftData  
@testable import Pault  
  
struct TagTests {  
  
    @Test func tagInitializesWithDefaults() async throws {  
        let tag = Tag(name: "work")  
  
        #expect(tag.name == "work")  
        #expect(tag.color == "blue")  
        #expect(tag.id != UUID(uuidString: "00000000-0000-0000-0000-000000000000"))  
    }  
  
    @Test func tagInitializesWithCustomColor() async throws {  
        let tag = Tag(name: "urgent", color: "red")  
  
        #expect(tag.name == "urgent")  
        #expect(tag.color == "red")  
    }  
}
```

Step 2: Run test to verify it fails

Run: `xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "(Test|error|FAIL|PASS)"`

Expected: FAIL - "Cannot find 'Tag' in scope"

Step 3: Write minimal implementation

Create Pault/Tag.swift:

```
//
// Tag.swift
// Pault
//

import Foundation
import SwiftData

@Model
final class Tag {
    var id: UUID
    var name: String
    var color: String
    var createdAt: Date
    @Relationship(inverse: \Prompt.tags) var prompts: [Prompt]?

    init(id: UUID = UUID(), name: String, color: String = "blue", createdAt: Date = Date())
    {
        self.id = id
        self.name = name
        self.color = color
        self.createdAt = createdAt
    }
}
```

Step 4: Run test to verify it passes

Run: `xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "(Test|error|FAIL|PASS)"`

Expected: Tests fail because `Prompt.tags` doesn't exist yet. This is expected—continue to next task.

Step 5: Commit

```
git add Pault/Tag.swift PaultTests/TagTests.swift
git commit -m "feat: add Tag model with name, color, and prompts relationship"
```

Task 2: Update Prompt Model for Tags**Files:**

- Modify: Pault/Prompt.swift
- Test: PaultTests/PromptTagTests.swift

Step 1: Write the failing test

Create PaultTests/PromptTagTests.swift:

```
//
// PromptTagTests.swift
// PaultTests
//

import Testing
import SwiftData
@testable import Pault

struct PromptTagTests {

    @Test func promptCanHaveMultipleTags() async throws {
        let prompt = Prompt(title: "Test", content: "Content")
        let tag1 = Tag(name: "work")
        let tag2 = Tag(name: "email")

        prompt.tags = [tag1, tag2]

        #expect(prompt.tags?.count == 2)
        #expect(prompt.tags?.contains(where: { $0.name == "work" }) == true)
        #expect(prompt.tags?.contains(where: { $0.name == "email" }) == true)
    }

    @Test func promptStartsWithNoTags() async throws {
        let prompt = Prompt(title: "Test", content: "Content")

        #expect(prompt.tags == nil || prompt.tags?.isEmpty == true)
    }
}
```

Step 2: Run test to verify it fails

Run: `xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "(Test|error|FAIL|PASS)"`

Expected: FAIL - "Value of type 'Prompt' has no member 'tags'"

Step 3: Write minimal implementation

Modify Pault/Prompt.swift to replace category with tags:

```
//
// Prompt.swift
// Pault
//
// Created by Jonathan Hines Dumitru on 12/16/25.
//

import Foundation
import SwiftData

@Model
final class Prompt {
    var id: UUID
    var title: String
    var content: String
    var isFavorite: Bool
    var isArchived: Bool
    @Relationship var tags: [Tag]?
    var createdAt: Date
}
```

```

var updatedAt: Date

init(id: UUID = UUID(), title: String, content: String, isFavorite: Bool = false,
isArchived: Bool = false, createdAt: Date = Date(), updatedAt: Date = Date(), tags:
[Tag]? = nil) {
    self.id = id
    self.title = title
    self.content = content
    self.isFavorite = isFavorite
    self.isArchived = isArchived
    self.createdAt = createdAt
    self.updatedAt = updatedAt
    self.tags = tags
}
}

```

Step 4: Run test to verify it passes

Run: `xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | grep -E "(Test|error|FAIL|PASS)"`

Expected: PASS for PromptTagTests (build may fail elsewhere due to Category references)

Step 5: Commit

```

git add Pault/Prompt.swift PaultTests/PromptTagTests.swift
git commit -m "feat: replace category with tags (many-to-many) on Prompt model"

```

Task 3: Update Schema and Remove Category

Files:

- Modify: Pault/PaultApp.swift
- Delete: Pault/Category.swift

Step 1: Update PaultApp.swift schema

Replace `Category.self` with `Tag.self`:

```

//
// PaultApp.swift
// Pault
//
// Created by Jonathan Hines Dumitru on 12/16/25.
//

import SwiftUI
import SwiftData

@main
struct PaultApp: App {
    var sharedModelContainer: ModelContainer = {
        let schema = Schema([
            Prompt.self,
            Tag.self,
        ])
        let persistentConfig = ModelConfiguration(schema: schema, isStoredInMemoryOnly: false)
    }()
}

```

```

do {
    return try ModelContainer(for: schema, configurations: [persistentConfig])
} catch {
    print("SwiftData ModelContainer load failed: \(error). Falling back to in-memory store.")
    let memoryConfig = ModelConfiguration(schema: schema, isStoredInMemoryOnly: true)
    do {
        return try ModelContainer(for: schema, configurations: [memoryConfig])
    } catch {
        fatalError("Failed to create in-memory ModelContainer fallback: \(error)")
    }
}
}()

var body: some Scene {
    WindowGroup {
        ContentView()
    }
    .windowResizability(.contentSize)
    .defaultSize(width: 900, height: 600)
    .modelContainer(sharedModelContainer)
}
}

```

Step 2: Delete Category.swift

```
rm Pault/Category.swift
```

Step 3: Build to verify schema compiles

Run: `xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -20`

Expected: Build fails due to Category references in ContentView—this is expected.

Step 4: Commit

```

git add Pault/PaultApp.swift
git rm Pault/Category.swift
git commit -m "feat: update schema to use Tag instead of Category"

```

Phase 2: Tag Pill Component

Task 4: Create TagPillView

Files:

- Create: Pault/TagPillView.swift

Step 1: Create the component

Create Pault/TagPillView.swift:

```

//
// TagPillView.swift
// Pault
//

```

```
import SwiftUI

struct TagPillView: View {
    let name: String
    let color: String
    var isSmall: Bool = false
    var onTap: (() -> Void)? = nil
    var onRemove: (() -> Void)? = nil

    private var pillColor: Color {
        switch color {
        case "blue": return .blue
        case "purple": return .purple
        case "pink": return .pink
        case "red": return .red
        case "orange": return .orange
        case "yellow": return .yellow
        case "green": return .green
        case "teal": return .teal
        case "gray": return .gray
        default: return .blue
        }
    }

    var body: some View {
        HStack(spacing: 4) {
            Text("#\(name)")
                .font(isSmall ? .caption2 : .caption)
                .fontWeight(.medium)

            if let onRemove = onRemove {
                Button(action: onRemove) {
                    Image(systemName: "xmark")
                        .font(.system(size: isSmall ? 8 : 10, weight: .bold))
                }
                .buttonStyle(.plain)
            }
        }
        .padding(.horizontal, isSmall ? 6 : 8)
        .padding(.vertical, isSmall ? 2 : 4)
        .background(pillColor.opacity(0.2))
        .foregroundColor(pillColor)
        .clipShape(Capsule())
        .contentShape(Capsule())
        .onTapGesture {
            onTap?()
        }
    }
}

struct TagPillsView: View {
    let tags: [Tag]
    var maxVisible: Int = 2
    var isSmall: Bool = false
    var onTagTap: ((Tag) -> Void)? = nil

    var body: some View {
        HStack(spacing: 4) {
```

```

ForEach(tags.prefix(maxVisible)) { tag in
    TagPillView(name: tag.name, color: tag.color, isSmall: isSmall) {
        onTapTap?(tag)
    }
}

if tags.count > maxVisible {
    Text("+\\(tags.count - maxVisible)")
    .font(isSmall ? .caption2 : .caption)
    .foregroundColor(.secondary)
}
}
}
}

#Preview {
    VStack(spacing: 16) {
        TagPillView(name: "work", color: "blue")
        TagPillView(name: "urgent", color: "red", isSmall: true)
        TagPillView(name: "email", color: "green", onRemove: {})
    }
    .padding()
}

```

Step 2: Build to verify it compiles

Run: `xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -10`

Expected: Build may still fail due to ContentView issues, but TagPillView should compile.

Step 3: Commit

```

git add Pault/TagPillView.swift
git commit -m "feat: add TagPillView and TagPillsView components"

```

Phase 3: Sidebar

Task 5: Create SidebarView

Files:

- Create: Pault/SidebarView.swift

Step 1: Create the sidebar component

Create Pault/SidebarView.swift:

```

//
// SidebarView.swift
// Pault
//

import SwiftUI
import SwiftData

enum SidebarFilter: Hashable {
    case all

```



```

case recent
case archived
case tag(Tag)
}

struct SidebarView: View {
    @Query(sort: [SortDescriptor(\Prompt.updatedAt, order: .reverse)]) private var
    allPrompts: [Prompt]
    @Query(sort: [SortDescriptor(\Tag.name, order: .forward)]) private var allTags: [Tag]

    @Binding var selectedPrompt: Prompt?
    @Binding var selectedFilter: SidebarFilter
    @Binding var searchText: String

    var onDelete: ((Prompt) -> Void)?
    var onToggleFavorite: ((Prompt) -> Void)?
    var onToggleArchive: ((Prompt) -> Void)?
    var onCopy: ((Prompt) -> Void)?

    private var filteredPrompts: [Prompt] {
        var prompts = allPrompts

        // Apply filter
        switch selectedFilter {
        case .all:
            prompts = prompts.filter { !$0.isArchived }
        case .recent:
            prompts = Array(prompts.filter { !$0.isArchived }.prefix(10))
        case .archived:
            prompts = prompts.filter { $0.isArchived }
        case .tag(let tag):
            prompts = prompts.filter { $0.tags?.contains(where: { $0.id == tag.id }) == true &&
                !$0.isArchived }
        }

        // Apply search
        if !searchText.isEmpty {
            let query = searchText.lowercased()
            prompts = prompts.filter {
                $0.title.lowercased().contains(query) ||
                $0.content.lowercased().contains(query) ||
                $0.tags?.contains(where: { $0.name.lowercased().contains(query) }) == true
            }
        }

        return prompts
    }

    var body: some View {
        VStack(spacing: 0) {
            // Search field
            HStack {
                Image(systemName: "magnifyingglass")
                    .foregroundColor(.secondary)
                TextField("Search", text: $searchText)
                    .textFieldStyle(.plain)
                if !searchText.isEmpty {
                    Button(action: { searchText = "" }) {

```

```

Image(systemName: "xmark.circle.fill")
.foregroundStyle(.secondary)
}
.buttonStyle(.plain)
}
}
.padding(8)
.background(.quaternary)
.clipShape(RoundedRectangle(cornerRadius: 8))
.padding(.horizontal, 12)
.padding(.top, 12)
.padding(.bottom, 8)

// Filters
VStack(spacing: 2) {
FilterRow(title: "Recently Used", icon: "clock", isSelected: selectedFilter == .recent)
{
selectedFilter = .recent
}
FilterRow(title: "All Prompts", icon: "doc.text", isSelected: selectedFilter == .all) {
selectedFilter = .all
}
FilterRow(title: "Archived", icon: "archivebox", isSelected: selectedFilter ==
.archived) {
selectedFilter = .archived
}
}
.padding(.horizontal, 8)

Divider()
.padding(.vertical, 8)

// Prompt list
List(selection: $selectedPrompt) {
ForEach(filteredPrompts) { prompt in
PromptRowView(prompt: prompt) {
onToggleFavorite?(prompt)
} onTap: { tag in
selectedFilter = .tag(tag)
}
}
.tag(prompt)
.contextMenu {
Button("Copy", systemImage: "doc.on.doc") { onCopy?(prompt) }
Button(prompt.isFavorite ? "Unfavorite" : "Favorite", systemImage: "star") {
onToggleFavorite?(prompt) }
Button(prompt.isArchived ? "Unarchive" : "Archive", systemImage: "archivebox") {
onToggleArchive?(prompt) }
Divider()
Button("Delete", systemImage: "trash", role: .destructive) { onDelete?(prompt) }
}
}
}
.listStyle(.sidebar)
.scrollContentBackground(.hidden)
}
.frame(minWidth: 220)
}
}

```

```
private struct FilterRow: View {
    let title: String
    let icon: String
    let isSelected: Bool
    let action: () -> Void

    var body: some View {
        Button(action: action) {
            HStack {
                Image(systemName: icon)
                    .frame(width: 20)
                Text(title)
                Spacer()
            }
            .padding(.vertical, 6)
            .padding(.horizontal, 8)
            .background(isSelected ? Color.accentColor.opacity(0.2) : Color.clear)
            .clipShape(RoundedRectangle(cornerRadius: 6))
        }
        .buttonStyle(.plain)
    }
}

private struct PromptRowView: View {
    let prompt: Prompt
    var onFavoriteTap: (() -> Void)?
    var onTagTap: ((Tag) -> Void)?

    private var displayTitle: String {
        if !prompt.title.isEmpty {
            return prompt.title
        }
        let preview = prompt.content.prefix(30)
        return preview.isEmpty ? "Untitled" : String(preview)
    }

    var body: some View {
        HStack(alignment: .top, spacing: 8) {
            VStack(alignment: .leading, spacing: 4) {
                HStack(spacing: 6) {
                    Text(displayTitle)
                        .font(.headline)
                        .lineLimit(1)
                }

                if prompt.isFavorite {
                    Image(systemName: "star.fill")
                        .font(.caption)
                        .foregroundColor(.yellow)
                }
            }

            if let tags = prompt.tags, !tags.isEmpty {
                TagPillsView(tags: tags, maxVisible: 2, isSmall: true, onTagTap: onTagTap)
            }
        }

        Spacer(minLength: 0)
    }
}
```

```

}
.padding(.vertical, 4)
}
}

```

Step 2: Build to verify it compiles

Run: `xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -10`

Step 3: Commit

```

git add Pault/SidebarView.swift
git commit -m "feat: add SidebarView with filters, search, and prompt list"

```

Phase 4: Inspector Panel

Task 6: Create InspectorView

Files:

- Create: Pault/InspectorView.swift

Step 1: Create the inspector component

Create Pault/InspectorView.swift:

```

//
// InspectorView.swift
// Pault
//

import SwiftUI
import SwiftData

struct InspectorView: View {
    @Environment(\.modelContext) private var modelContext
    @Query(sort: [SortDescriptor(\Tag.name, order: .forward)]) private var allTags: [Tag]

    @Bindable var prompt: Prompt
    @State private var newTagName: String = ""
    @State private var showingTagPicker: Bool = false

    private let tagColors = ["blue", "purple", "pink", "red", "orange", "yellow", "green",
    "teal", "gray"]

    var body: some View {
        VStack(alignment: .leading, spacing: 16) {
            // Tags section
            VStack(alignment: .leading, spacing: 8) {
                Text("Tags")
                .font(.headline)
                .foregroundColor(.secondary)

                FlowLayout(spacing: 6) {
                    ForEach(prompt.tags ?? []) { tag in
                        TagPillView(name: tag.name, color: tag.color, onRemove: {

```

```

removeTag(tag)
})
}

Button(action: { showingTagPicker.toggle() }) {
Image(systemName: "plus")
.font(.caption)
.padding(6)
.background(Color.secondary.opacity(0.2))
.clipShape(Circle())
}
.buttonStyle(.plain)
.popover(isPresented: $showingTagPicker) {
TagPickerView(
existingTags: allTags,
selectedTags: prompt.tags ?? [],
onSelect: { tag in
addTag(tag)
},
onCreate: { name, color in
createAndAddTag(name: name, color: color)
}
)
.frame(width: 200, height: 300)
}
}
}

Divider()

// Favorite toggle
HStack {
Text("Favorite")
.foregroundColor(.secondary)
Spacer()
Button(action: { prompt.isFavorite.toggle() }) {
Image(systemName: prompt.isFavorite ? "star.fill" : "star")
.foregroundColor(prompt.isFavorite ? .yellow : .secondary)
}
.buttonStyle(.plain)
}

Divider()

// Dates
VStack(alignment: .leading, spacing: 4) {
HStack {
Text("Created")
.foregroundColor(.secondary)
Spacer()
Text(prompt.createdAt.formatted(date: .abbreviated, time: .shortened))
.font(.caption)
}
HStack {
Text("Modified")
.foregroundColor(.secondary)
Spacer()
Text(prompt.updatedAt.formatted(date: .abbreviated, time: .shortened))
}
}

```

```

        .font(.caption)
    }
}

Divider()

// Archive button
Button(action: { prompt.isArchived.toggle() }) {
    Label(prompt.isArchived ? "Unarchive" : "Archive", systemImage: "archivebox")
}
.buttonStyle(.plain)
.foregroundColor(prompt.isArchived ? .blue : .secondary)

Spacer()
}
.padding()
.frame(width: 220)
.background(.regularMaterial)
}

private func addTag(_ tag: Tag) {
    if prompt.tags == nil {
        prompt.tags = []
    }
    if !(prompt.tags?.contains(where: { $0.id == tag.id }) ?? false) {
        prompt.tags?.append(tag)
    }
    showingTagPicker = false
}

private func removeTag(_ tag: Tag) {
    prompt.tags?.removeAll(where: { $0.id == tag.id })
}

private func createAndAddTag(name: String, color: String) {
    let tag = Tag(name: name, color: color)
    modelContext.insert(tag)
    addTag(tag)
}

private struct TagPickerView: View {
    let existingTags: [Tag]
    let selectedTags: [Tag]
    let onSelect: (Tag) -> Void
    let onCreate: (String, String) -> Void

    @State private var newTagName: String = ""
    @State private var selectedColor: String = "blue"

    private let colors = ["blue", "purple", "pink", "red", "orange", "yellow", "green",
        "teal", "gray"]

    private var availableTags: [Tag] {
        existingTags.filter { tag in
            !selectedTags.contains(where: { $0.id == tag.id })
        }
    }
}

```

```
var body: some View {
VStack(alignment: .leading, spacing: 12) {
Text("Add Tag")
.font(.headline)

if !availableTags.isEmpty {
ScrollView {
VStack(spacing: 4) {
ForEach(availableTags) { tag in
Button(action: { onSelect(tag) }) {
HStack {
TagPillView(name: tag.name, color: tag.color, isSmall: true)
Spacer()
}
}
.buttonStyle(.plain)
}
}
.frame(maxHeight: 120)

Divider()
}

Text("Create New")
.font(.subheadline)
.foregroundColor(.secondary)

TextField("Tag name", text: $newTagName)
.textFieldStyle(.roundedBorder)

HStack(spacing: 4) {
ForEach(colors, id: \.self) { color in
Circle()
.fill(colorFor(color))
.frame(width: 20, height: 20)
.overlay(
Circle()
.stroke(Color.primary, lineWidth: selectedColor == color ? 2 : 0)
)
.onTapGesture {
selectedColor = color
}
}
}

Button("Create") {
let trimmed = newTagName.trimmingCharacters(in: .whitespacesAndNewlines)
guard !trimmed.isEmpty else { return }
onCreate(trimmed, selectedColor)
newTagName = ""
}
.disabled(newTagName.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty)
}
.padding()
}
```

```
private func colorFor(_ name: String) -> Color {
    switch name {
    case "blue": return .blue
    case "purple": return .purple
    case "pink": return .pink
    case "red": return .red
    case "orange": return .orange
    case "yellow": return .yellow
    case "green": return .green
    case "teal": return .teal
    case "gray": return .gray
    default: return .blue
    }
}

struct FlowLayout: Layout {
    var spacing: CGFloat = 8

    func sizeThatFits(proposal: ProposedViewSize, subviews: Subviews, cache: inout ()) ->
    CGSize {
        let result = arrange(proposal: proposal, subviews: subviews)
        return result.size
    }

    func placeSubviews(in bounds: CGRect, proposal: ProposedViewSize, subviews: Subviews,
    cache: inout ()) {
        let result = arrange(proposal: proposal, subviews: subviews)
        for (index, position) in result.positions.enumerated() {
            subviews[index].place(at: CGPoint(x: bounds.minX + position.x, y: bounds.minY +
            position.y), proposal: .unspecified)
        }
    }

    private func arrange(proposal: ProposedViewSize, subviews: Subviews) -> (size: CGSize,
    positions: [CGPoint]) {
        let maxWidth = proposal.width ?? .infinity
        var positions: [CGPoint] = []
        var currentX: CGFloat = 0
        var currentY: CGFloat = 0
        var lineHeight: CGFloat = 0

        for subview in subviews {
            let size = subview.sizeThatFits(.unspecified)

            if currentX + size.width > maxWidth && currentX > 0 {
                currentX = 0
                currentY += lineHeight + spacing
                lineHeight = 0
            }

            positions.append(CGPoint(x: currentX, y: currentY))
            lineHeight = max(lineHeight, size.height)
            currentX += size.width + spacing
        }

        return (CGSize(width: maxWidth, height: currentY + lineHeight), positions)
    }
}
```



```
}
```

Step 2: Build to verify it compiles

Run: `xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -10`

Step 3: Commit

```
git add Pault/InspectorView.swift
git commit -m "feat: add InspectorView with tags, favorite, dates, and archive"
```

Phase 5: Detail View

Task 7: Rewrite PromptDetailView

Files:

- Modify: `Pault/PromptDetailView.swift`

Step 1: Rewrite the detail view

Replace `Pault/PromptDetailView.swift`:

```
//
// PromptDetailView.swift
// Pault
//
// Created by Jonathan Hines Dumitru on 12/16/25.
//

import SwiftUI
import SwiftData
import Combine

#if os(macOS)
import AppKit
#endif

struct PromptDetailView: View {
    @Environment(\.modelContext) private var modelContext
    @Bindable var prompt: Prompt

    @Binding var showInspector: Bool
    @State private var saveTask: Task<Void, Never>?

    var body: some View {
        HStack(spacing: 0) {
            // Main editor
            VStack(alignment: .leading, spacing: 0) {
                // Title field
                TextField("Untitled", text: $prompt.title)
                    .font(.title2)
                    .fontWeight(.semibold)
                    .textFieldStyle(.plain)
                    .padding(.horizontal, 20)
                    .padding(.top, 20)
            }
        }
    }
}
```

```

.padding(.bottom, 12)
.onChange(of: prompt.title) { _, _ in
debouncedSave()
}

// Content editor
TextEditor(text: $prompt.content)
.font(.body)
.scrollContentBackground(.hidden)
.padding(.horizontal, 16)
.padding(.bottom, 16)
.onChange(of: prompt.content) { _, _ in
debouncedSave()
}
}
.frame(maxWidth: .infinity, maxHeight: .infinity)

// Inspector panel
if showInspector {
Divider()
InspectorView(prompt: prompt)
.transition(.move(edge: .trailing))
}
}
.animation(.easeInOut(duration: 0.2), value: showInspector)
.overlay(alignment: .bottomTrailing) {
Button(action: { showInspector.toggle() }) {
Image(systemName: "info.circle")
.font(.title2)
.foregroundColor(showInspector ? .blue : .secondary)
.padding(12)
}
.buttonStyle(.plain)
.keyboardShortcut("i", modifiers: .command)
.help("Toggle Inspector (⌘I)")
}
.onChange(of: prompt.isFavorite) { _, _ in
debouncedSave()
}
.onChange(of: prompt.isArchived) { _, _ in
debouncedSave()
}
.onChange(of: prompt.tags) { _, _ in
debouncedSave()
}
}

private func debouncedSave() {
saveTask?.cancel()
saveTask = Task {
try? await Task.sleep(for: .milliseconds(500))
guard !Task.isCancelled else { return }
await MainActor.run {
prompt.updatedAt = Date()
try? modelContext.save()
}
}
}
}

```

```

}

struct EmptyDetailView: View {
    var body: some View {
        VStack(spacing: 12) {
            Image(systemName: "doc.text")
                .font(.system(size: 48))
                .foregroundColor(.tertiary)
            Text("Select a prompt or press ■N to create one")
                .foregroundColor(.secondary)
        }
        .frame(maxWidth: .infinity, maxHeight: .infinity)
    }
}

```

Step 2: Build to verify it compiles

Run: `xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -10`

Step 3: Commit

```

git add Pault/PromptDetailView.swift
git commit -m "feat: rewrite PromptDetailView with always-editable fields and inspector toggle"

```

Phase 6: Main Content View

Task 8: Rewrite ContentView with NavigationSplitView

Files:

- Modify: Pault/ContentView.swift

Step 1: Rewrite ContentView

Replace Pault/ContentView.swift:

```

//
// ContentView.swift
// Pault
//
// Created by Jonathan Hines Dumitru on 12/16/25.
//

import SwiftUI
import SwiftData

#if os(macOS)
import AppKit
#endif

struct ContentView: View {
    @Environment(\.modelContext) private var modelContext
    @Query(sort: [SortDescriptor(\Prompt.updatedAt, order: .reverse)]) private var prompts:
    [Prompt]

```

```

@State private var selectedPrompt: Prompt?
@State private var selectedFilter: SidebarFilter = .all
@State private var searchText: String = ""
@State private var showInspector: Bool = false
@State private var columnVisibility: NavigationSplitViewVisibility = .automatic

var body: some View {
    NavigationSplitView(columnVisibility: $columnVisibility) {
        SidebarView(
            selectedPrompt: $selectedPrompt,
            selectedFilter: $selectedFilter,
            searchText: $searchText,
            onDelete: deletePrompt,
            onToggleFavorite: toggleFavorite,
            onToggleArchive: toggleArchive,
            onCopy: copyPrompt
        )
        .navigationSplitViewColumnWidth(min: 220, ideal: 260, max: 320)
    } detail: {
        if let prompt = selectedPrompt {
            PromptDetailView(prompt: prompt, showInspector: $showInspector)
                .id(prompt.id)
        } else {
            EmptyDetailView()
        }
    }
    .navigationSplitViewStyle(.balanced)
    .toolbar {
        ToolbarItem(placement: .navigation) {
            Button(action: { toggleSidebar() }) {
                Image(systemName: "sidebar.left")
            }
        }
        .keyboardShortcut("0", modifiers: .command)
        .help("Toggle Sidebar (0)")
    }

    ToolbarItemGroup(placement: .primaryAction) {
        Button(action: copySelectedPrompt) {
            Image(systemName: "doc.on.doc")
        }
        .keyboardShortcut("c", modifiers: .command)
        .disabled(selectedPrompt == nil)
        .help("Copy Prompt Content (C)")

        Button(action: createNewPrompt) {
            Image(systemName: "plus")
        }
        .keyboardShortcut("n", modifiers: .command)
        .help("New Prompt (N)")
    }
    .frame(minWidth: 700, minHeight: 500)
    .onDeleteCommand {
        if let prompt = selectedPrompt {
            deletePrompt(prompt)
        }
    }
}

```

```
private func toggleSidebar() {
  withAnimation {
    switch columnVisibility {
    case .detailOnly:
      columnVisibility = .all
    default:
      columnVisibility = .detailOnly
    }
  }
}

private func createNewPrompt() {
  let prompt = Prompt(title: "", content: "")
  modelContext.insert(prompt)
  try? modelContext.save()
  selectedPrompt = prompt
}

private func deletePrompt(_ prompt: Prompt) {
  if selectedPrompt?.id == prompt.id {
    selectedPrompt = nil
  }
  modelContext.delete(prompt)
  try? modelContext.save()
}

private func toggleFavorite(_ prompt: Prompt) {
  prompt.isFavorite.toggle()
  prompt.updatedAt = Date()
  try? modelContext.save()
}

private func toggleArchive(_ prompt: Prompt) {
  prompt.isArchived.toggle()
  prompt.updatedAt = Date()
  try? modelContext.save()
}

private func copyPrompt(_ prompt: Prompt) {
  #if os(macOS)
  NSPasteboard.general.clearContents()
  NSPasteboard.general.setString(prompt.content, forType: .string)
  #endif
}

private func copySelectedPrompt() {
  guard let prompt = selectedPrompt else { return }
  copyPrompt(prompt)
}

#Preview {
  ContentView()
  .modelContainer(for: [Prompt.self, Tag.self], inMemory: true)
}
```

Step 2: Build and run to verify

```
Run: xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -20
```

Expected: BUILD SUCCEEDED

Step 3: Commit

```
git add Pault/ContentView.swift
git commit -m "feat: rewrite ContentView with NavigationSplitView and unified toolbar"
```

Phase 7: Cleanup

Task 9: Delete Unused Files

Files:

- Delete: Pault/WindowSizeConstraints.swift

Step 1: Delete the file

```
rm Pault/WindowSizeConstraints.swift
```

Step 2: Build to verify nothing breaks

```
Run: xcodebuild build -scheme Pault -destination 'platform=macOS' 2>&1 | tail -10
```

Expected: BUILD SUCCEEDED

Step 3: Commit

```
git rm Pault/WindowSizeConstraints.swift
git commit -m "chore: remove WindowSizeConstraints (window is now resizable)"
```

Task 10: Update Preview in ContentView

Files:

- Verify: Pault/ContentView.swift

Step 1: Build and run the app

```
xcodebuild build -scheme Pault -destination 'platform=macOS' && open
~/Library/Developer/Xcode/DerivedData/Pault-*/Build/Products/Debug/Pault.app
```

Step 2: Manual verification

Verify:

- [] App launches without crash
- [] Sidebar shows with filters and prompt list
- [] Can create new prompt with Cmd+N
- [] Can select a prompt and see detail view
- [] Sidebar auto-hides when prompt selected (toggle with Cmd+0)
- [] Inspector toggles with Cmd+I
- [] Copy works with Cmd+C
- [] Tags can be added in inspector

- [] Changes auto-save

Step 3: Final commit

```
git add -A
git commit -m "feat: complete UI overhaul with NavigationSplitView, tags, and inspector"
```

Summary

| Phase | Tasks | Description |

|-----|-----|-----|

| 1 | 1-3 | Data model: Tag model, update Prompt, update schema |

| 2 | 4 | TagPillView component |

| 3 | 5 | SidebarView with filters and list |

| 4 | 6 | InspectorView with tag editing |

| 5 | 7 | PromptDetailView rewrite |

| 6 | 8 | ContentView with NavigationSplitView |

| 7 | 9-10 | Cleanup and verification |

Total: 10 tasks, ~30-45 minutes implementation time